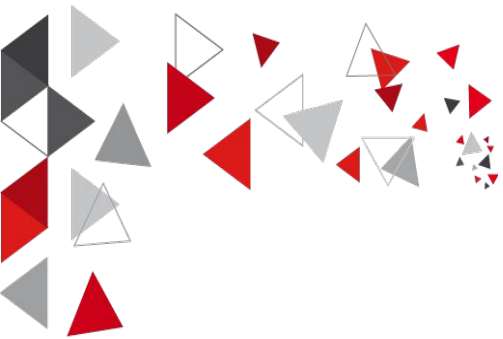
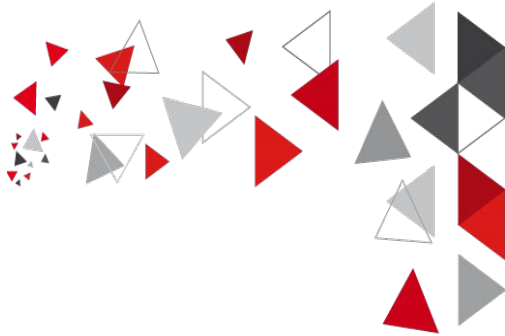


esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Conception Orienté Objet et Programmation Java

Chapitre 11 : MAP

Objectifs du chapitre

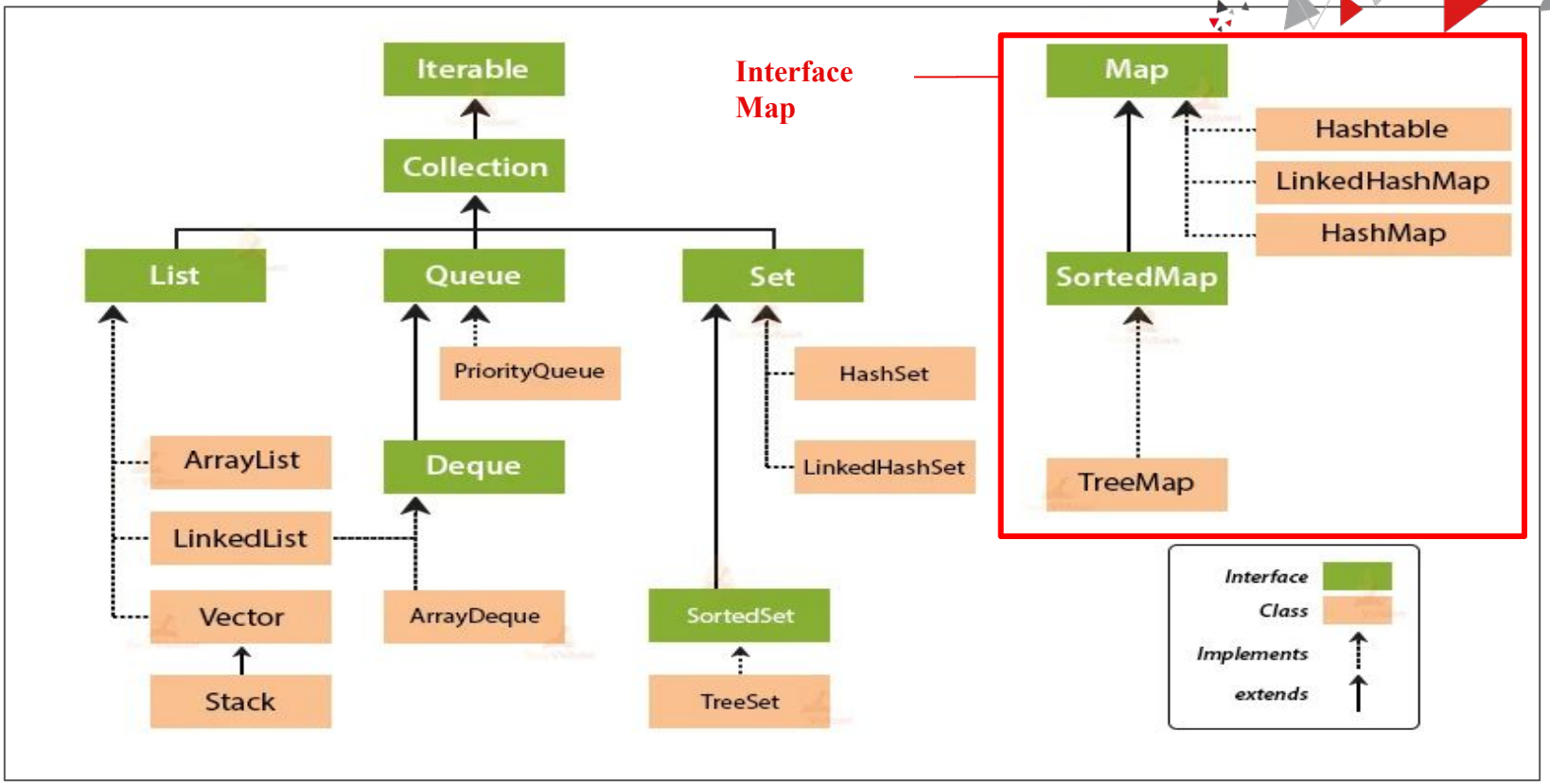


- Découvrir l'interface MAP , son architecture ainsi que les classes qui l'implémentent
- Savoir différencier entre HashMap et TreeMap

MAP



Architecture



MAP… Par définition

Une map est une structure qui associe une **clé** (Key) à une **valeur** (Value) (Map<K,V>).

On peut penser à la clé comme étant le mécanisme d'accès à la valeur correspondante dans la Map. **Key**  **Value**

La clé est utilisée pour indexer la valeur, permettant ainsi une recherche rapide d'une valeur donnée en utilisant sa clé.

Dans une Map la clé est **unique**, contrairement à la valeur qui peut être associée à plusieurs clés.

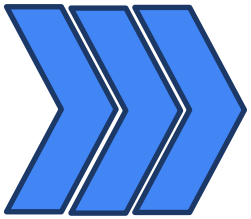
NB: La redéfinition des méthodes hashCode et equals est nécessaire pour la clé

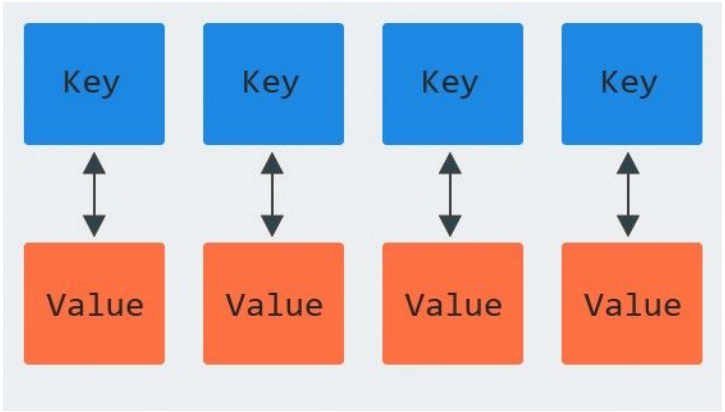
Notez Bien!....

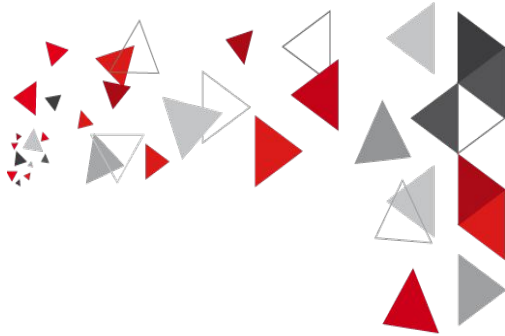
Map n’est pas une Collection !!!

La raison pour laquelle **Map** n’est pas une **Collection** est liée à la nature distincte de ces deux structures de données :

- Une **Collection** représente un ensemble d'éléments, où chaque élément est une entité individuelle.
- Une **Map** est une structure de données qui stocke des paires **clé-valeur**, où chaque élément correspond à une association entre une clé et une valeur.





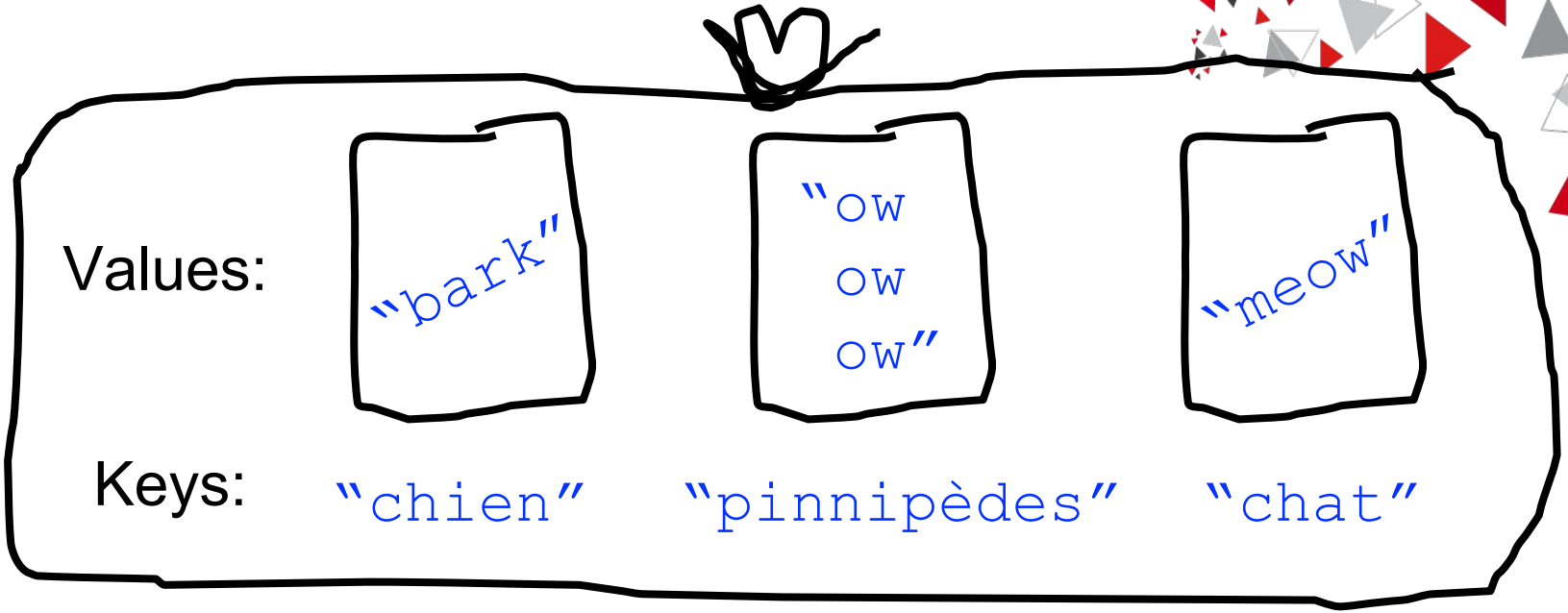


L’interface **Map** peut être implémentée avec les classes:

- **HashMap**: **ne garantie pas** l'ordre d'insertion des éléments lors de parcours.

- **TreeMap**: stocke les éléments triés selon leurs **clés**.

MAP… Un simple exemple



[key = “chien”, value = “bark”]
[key=“chat”, value=“meow”]
[key=“pinnipèdes”, value=“ow ow ow”]

MAP… Quelques méthodes utiles

Méthode	Explication	Paramètres	Type de retour
put(K key, V value)	Ajoute une paire clé-valeur à la Map ou remplace la valeur existante pour une clé donnée. La méthode renvoie la valeur précédemment associée à la clé donnée, ou null si aucune valeur n'était associée à cette clé.	K : type de la clé, V : type de la valeur	V : la valeur précédemment associée à la clé donnée, ou null
get(K key)	Récupère la valeur associée à la clé donnée, ou null si la clé n'existe pas dans la Map.	K : la clé	V : la valeur associée à la clé donnée, ou null
remove(K key)	Supprime la paire clé-valeur associée à la clé donnée. La méthode renvoie la valeur associée à la clé donnée, ou null si la clé n'existe pas dans la Map.	K : la clé	V : la valeur associée à la clé donnée, ou null
containsKey(K key)	Vérifie si la Map contient la clé donnée. La méthode renvoie true si la Map contient la clé, et false sinon.	K : la clé	boolean : true si la Map contient la clé, false sinon
containsValue(V value)	Vérifie si la Map contient la valeur donnée. La	V : la valeur	boolean : true si la Map

MAP… Quelques méthodes utiles

Méthode	Explication	Paramètres	Type de retour
size()	Renvoie le nombre de paires clé-valeur dans la Map.	-	int : le nombre de paires clé-valeur dans la Map
isEmpty()	Vérifie si la Map est vide. La méthode renvoie true si la Map ne contient aucune paire clé-valeur, et false sinon.	-	boolean : true si la Map est vide, false sinon
keySet()	Renvoie un ensemble contenant toutes les clés de la Map. La méthode renvoie un objet Set qui peut être utilisé pour parcourir les clés de la Map.	-	Set<K> : un ensemble contenant toutes les clés de la Map
values()	Renvoie une collection contenant toutes les valeurs de la Map. La méthode renvoie un objet Collection qui peut être utilisé pour parcourir les valeurs de la Map.	-	Collection<V> : une collection contenant toutes les valeurs de la Map
entrySet()	Renvoie un ensemble contenant toutes les paires clé-valeur de la Map. La méthode	-	Set<Map.Entry<K, V>> : un ensemble contenant toutes les

Map.Entry<K,V>

L'interface **Map** contient une interface interne **Map.Entry<K,V>** est pour représenter une couple clé-valeur.

Cette interface définit 3 méthodes:

- V getValue()
- K getKey()
- V setValue(V value)

La méthode **entrySet()** de Map renvoie un objet de type **Set<Map.Entry<K, V>>**

HashMap



HashMap… par définition

La classe HashMap présente plusieurs caractéristiques :

- Elle permet l'utilisation de la valeur null comme clé et comme valeur.

- Elle ne garantit aucun ordre lors du parcours des éléments.

Map : exemple (1/4)

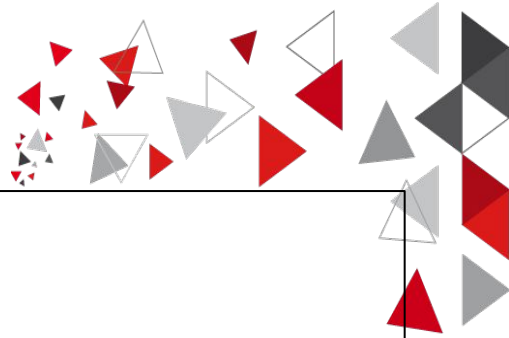
```java
// Creating a HashMap
Map<String, Integer> studentScores = new HashMap<>();

// Adding key-value pairs
studentScores.put("Alice", 12);
studentScores.put("Bob", 15);
studentScores.put("Charlie", 14);
studentScores.put("David", 20);
```

```java
// Getting the value for a specific key
String studentName = "Bob";
int score = studentScores.get(studentName);
System.out.println(studentName + "'s score: " + score); // Output: Bob's score: 15
```

```java
// Checking if a key is present (equals(Object o) override needed)
String newStudent = "Eva";
if (studentScores.containsKey(newStudent)) {
    System.out.println(newStudent + " is in the map.");
} else {
    System.out.println(newStudent + " is not in the map.");
}
// Output: Eva is not in the map.
```

Map : exemple (2/4)



```
// Removing a key-value pair
String removeStudent = "Charlie";
studentScores.remove(removeStudent);
System.out.println(removeStudent + " has been removed.");
// Output: Charlie has been removed.
```

```
// Displaying the map (toString() override needed for both Key and Value)
System.out.println("Student Scores: " + studentScores);
// Output: Student Scores: {Bob=15, Alice=12, David=20}
```

```
// Looping and displaying keys
Set<String> keys = studentScores.keySet();
Iterator<String> iterator = keys.iterator();
while (iterator.hasNext()) {
    System.out.println("Key: " + iterator.next());
}
```

```
// Output:
Key: Bob
Key: Alice
Key: David
```

Map : exemple (3/4)

```java
// Looping and displaying values
Collection<Integer> values = studentScores.values();
for (Integer value : values) {
    System.out.println("Value: " + value);
}
```

```
// Output:
Value: 15
Value: 12
Value: 20
```

```java
// Looping and displaying both keys and values
for (Map.Entry<String, Integer> entry : studentScores.entrySet()){
    System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

```
// Output:
Bob: 15
Alice: 12
David: 20
```

```java
// Checking the size of the map
System.out.println("Number of students: " + studentScores.size());
// Output: Number of students: 3
```

Map : exemple (4/4)

```
// Checking the size of the map
System.out.println("Number of students: " + studentScores.size());
// Output: Number of students: 3


// Clearing the map
studentScores.clear();


// Check if the map is empty
System.out.println("Is the map empty? " + studentScores.isEmpty());
// Output: Is the map empty? true
```

TreeMap



TreeMap… par définition

La classe **TreeMap**, ajoutée à **Java 1.2**, est une Map qui stocke des éléments de manière triée.

Les éléments de la collection sont triés selon l'ordre naturel de leur clé (s'ils implémentent l'interface Comparable) ou en utilisant une instance de type Comparator fournie au constructeur de la collection.

Elle implémente les interfaces **Map** et **SortedMap**.

TreeMap : exemple (1/2)

```
// Comparator for sorting by length of students names
Comparator<String> nameLengthComparator = new Comparator<String>() {
    @Override
    public int compare(String o1, String o2) {
        return o1.length() - o2.length();
    }
};

// Comparator for sorting Alphabetically
Comparator<String> alphabeticComparator = new Comparator<String>() {
    @Override
    public int compare(String o1, String o2) {
        return o1.compareTo(o2);
    }
};

// Creating a TreeMap with a combined comparator (first by length, then
Alphabetically)
Map<String, Integer> studentScores = new
TreeMap<>(nameLengthComparator.thenComparing(alphabeticComparator));
```

TreeMap : exemple (2/2)

```
// Adding key-value pairs to the TreeMap
studentScores.put("David", 20);
studentScores.put("Charlie", 14);
studentScores.put("Bob", 15);
studentScores.put("Alice", 12);

// Displaying the sorted map
System.out.println("Student Scores (first by length, then Alphabetically): " +
studentScores);

// Output:

Student Scores (first by length, then Alphabetically):

{Bob=15, Alice=12, David=20, Charlie=14}
```

LIST, SET ou MAP … Quoi utiliser?

- Si vous ne souhaitez pas que les valeurs soient dupliquées dans la base de données, Set doit être votre premier choix, car toutes ses classes n'autorisent pas les doublons.

- Si des opérations de recherche fréquentes basées sur les valeurs d'index sont nécessaires, alors List (ArrayList) constitue un meilleur choix.

- S'il est nécessaire de maintenir l'ordre d'insertion, List est également préférée.

- Si l'exigence est d'avoir un mappage clés/valeurs, alors Map est votre meilleur choix.

LIST Vs. SET Vs. MAP

	List	Set	Map
Duplication	List permet les éléments en double. N'importe quel nombre d'éléments en double peut être inséré dans la liste sans affecter les mêmes valeurs existantes et leurs index.	Set n'autorise pas les doublons. Set et toutes les classes qui implémentent l'interface Set doivent avoir des éléments uniques.	Map stocke les éléments sous forme de paire clé/valeur. Map n'autorise pas les clés en double, par contre il autorise les valeurs en double.
Valeurs nulles	List autorise n'importe quel nombre de valeurs NULL.	Set autorise au maximum une seule valeur NULL.	Map peut avoir au maximum une seule clé NULL et un nombre quelconque de valeurs NULL.
Ordre	List et toutes les classes implémentant l'interface List maintient l'ordre d'insertion.	Set ne maintient aucun ordre; encore il y a quelques classes implémentant l'interface Set, trient les éléments dans un ordre tel que LinkedHashSet maintient les éléments dans l'ordre d'insertion.	Pareil à Set, Map ne stocke pas les éléments dans un ordre, mais il y a quelques classes implémentant l'interface Map, stockent les éléments dans un ordre. Par exemple TreeMap trie la map dans l'ordre croissant selon les clés et LinkedHashMap trie les éléments dans l'ordre d'insertion, l'ordre dans lequel ils ont été ajoutés à LinkedHashMap.
Les classes utilisés	ArrayList, LinkedList etc.	HashSet, LinkedHashSet, TreeSet, SortedSet etc.	HashMap, TreeMap, WeakHashMap, LinkedHashMap, IdentityHashMap, etc.

Merci pour votre attention

